Build your own research hub

AI assistants that help you prove theorems, and keep an honest record of what is actually proved

> Printed 15 October 2026. The latest version is online at https://ilpenzo.github.io/math-research-hub/.

A guide for mathematicians with no AI experience. You will need about thirty minutes to install and an hour to learn. No programming is required.

> **Quick start: the whole process on one screen**
>
> 1. Buy a plan: Claude Pro or above, or a ChatGPT plan that includes Codex.
> 2. **Windows only:** first set up WSL as described under Before you start. From then on use the **Ubuntu** app, never PowerShell or the plain Terminal app.
> 3. Open a terminal. On a Mac press Cmd+Space, type `Terminal` and press Return. On Windows open **Ubuntu** from the Start menu.
> 4. Paste the install command from Step 1 (Mac: Cmd+V; Ubuntu: Ctrl+Shift+V), press Return, and answer the questions. Return (Enter) accepts each suggestion.
> 5. Open a **new** terminal window. Type the start line the installer printed at the end, such as `cd ~/research-hub && claude`, and press Return. Use this same line every time you come back.
> 6. Log in. When Claude Code asks whether you trust the folder, press the **down arrow** to choose "Yes, I trust this folder", then press Return. Return on its own means "No".
> 7. Paste your first request and press Return: *Read START_HERE.md and AGENTS.md, then give me a ten-line tour of this hub and suggest what I should do first.*
>
> Each step is explained below. If something goes wrong, see Troubleshooting.

What this is

A **research hub** is a folder on your computer in which AI assistants work on your mathematics. They work under a fixed set of rules. These are the rules a careful collaborator would follow anyway, written down so that the assistants follow them every time.

Here is how it helps you prove theorems faster.

- **It never starts from zero.** Results, failed attempts and open questions are kept in files. Each new session reads them first, so you never re-explain your setup and the assistant never re-proposes last month's dead end.
- **It searches before it invents.** Before proposing anything, it searches your own earlier notes, TeX files and code, and the papers in your library. The closest prior result is often already in your notes.
- **It works on the right question.** Every investigation begins by naming the one missing implication that blocks the proof. Everything else serves that step.
- **It runs a small proof team.** The main session develops the argument and can hand narrow side questions to specialist helpers that work in parallel. One finds a theorem in the literature. One makes a bold attempt at a lemma. One hunts for a counterexample to a step. One audits the finished proof.
- **It checks itself properly.** A claimed proof is frozen and audited by a helper that did not write it and has not seen the author's enthusiasm. A citation counts only after the theorem has been checked against the actual paper.
- **It turns failure into information.** When an approach fails, the hub records exactly where it broke and what new idea would justify trying again.

What it does not do: it does not replace your judgement, it does not certify proofs, and it can be wrong. It makes errors easier to catch and keeps the record honest about what has been established.

Before you start

You need four things.

1. **A computer running macOS or Linux, or Windows with WSL.** WSL (Windows Subsystem for Linux) is a free part of Windows. To set it up once:
   i. Right-click the Start button and choose "Terminal (Admin)" (on Windows 10, "Windows PowerShell (Admin)"). If Windows asks whether to allow the app to make changes, click **Yes**.
   ii. Type `wsl --install` and press Enter. When it finishes, restart the computer.
   iii. Open **Ubuntu** from the Start menu. When it asks for a new user name, choose a short one in lowercase letters with no spaces, such as `jsmith`. Then choose a password. **Nothing appears on screen while you type a password; that is normal.** Write the password down.
   iv. From now on, do everything in this guide in the Ubuntu window, not in PowerShell or the Terminal app.

   On some computers WSL asks for the user name and password straight away, or opens Ubuntu by itself after the restart. Answer as in step 3: that is expected.

   If `wsl --install` reports that virtualization is not enabled, follow Microsoft's instructions.

2. **A paid plan for at least one AI assistant.**
   - **Claude Code** (from Anthropic) needs a Claude Pro, Max, Team or Enterprise plan, or an Anthropic API account. The free plan does not include it. Claude plans.
   - **Codex** (from OpenAI) needs a ChatGPT plan that includes Codex (Plus, Pro, Business, Edu or Enterprise), or an OpenAI API key. Which ChatGPT plans include Codex.

   One is enough. With both, each can review the other's proofs, and two different AI systems catch more errors than one. Prices and usage limits change, so check the current plans.

3. **A folder of your own earlier work** (optional but valuable): notes, TeX files, code, PDFs.
4. **A lemma you have already proved** and know well. You will use it in Step 3 to see how far to trust the assistant in your area.

Never used a terminal? Anthropic's short terminal guide explains how to open one and paste a command.

About the requests in this guide. The indented boxes are things to paste to the assistant, followed by Return. Replace each part in [square brackets], brackets included,

with your own words. “[name]” means your project’s title or its short folder name; either works.

Keys. “Return” is the key labelled Return on a Mac and Enter on most other keyboards. To paste into a terminal, press Cmd+V on a Mac, or Ctrl+Shift+V in Ubuntu.

Step 1. Install (about 15 minutes)

Open a terminal. A terminal is a window where you type commands.

- **Mac:** press Cmd+Space, type `Terminal`, press Return.
- **Windows:** open **Ubuntu** from the Start menu (after the WSL setup above).
- **Linux:** open the “Terminal” application.

Paste this line, then press Return:

```
curl -fsSL https://raw.githubusercontent.com/ilpenzo/math-research-hub/main/install.sh | bash
```

On a Mac that has never used developer tools, a window may appear saying that the command requires the “command line developer tools”. Click **Install** and wait until it finishes, which can take 10 to 20 minutes. Then paste the install line again and press Return.

This downloads the installer from github.com/ilpenzo/math-research-hub and runs it. If you prefer to read a script before running it, which is a good habit, download the repository, read `install.sh`, and run `bash install.sh`.

Answer the questions. They come in seven short sections. Press Return to accept the suggestion shown in [brackets]. For a yes/no question, `[Y/n]` means that Return answers yes; type `n` and press Return for no.

1. **About you.** Your name, used in notes and the changelog.
2. **Where the hub lives.** Return accepts a folder called `research-hub` in your home folder.
3. **Your first project.**
   - a title, such as “Rationality of cubic fourfolds”;
   - a short folder name: press Return to accept the suggestion;
   - two to five capital letters to label its results (`RCF-R001`, ...): press Return to accept the suggestion;
   - the main question in a sentence or two, or press Return to skip.

4. **Conventions.** Spend five minutes here: this is where your expertise enters. List the things in your field that must never be left implicit, pressing Return after each one. Examples: “base field C unless stated”, “smooth means smooth over the base field”, “vector bundle, not coherent sheaf”, “groups are finite unless stated”. Press Return on an empty line to finish. Every helper’s instructions repeat these, and you can revise them later.
5. **Your earlier work** (optional). Folders the assistants may search but never change.
    - **Mac:** drag the folder from a Finder window into the Terminal window, then press Return. If your Mac asks whether Terminal may access the folder, click **OK**.
    - **Windows:** in File Explorer, open the folder and click the address bar at the top so that it shows the full location (such as `C:\Users\Name\OneDrive\Documents\old-notes`). Copy it with Ctrl+C, paste it into Ubuntu with Ctrl+Shift+V, and press Return. The installer converts it.
    - If a folder is not found, the installer says so and lets you try again. Press Return on an empty line to finish or to skip. You can add folders later.
6. **AI assistants.** Here, do not just press Return: type the number that matches your plan, 1 (Claude Code), 2 (Codex) or 3 (both), and press Return. Then, only where they apply:
    - an offer to install a missing assistant with its official installer: answer yes;
    - on a Mac with Homebrew (a common tool for installing programs), an offer to install `pdftotext` so the assistants can search PDFs: answer yes. Without Homebrew this question does not appear;
    - if you chose Codex, an offer to mark the hub as trusted for Codex: answer yes. It backs up your Codex settings first.
7. **Version history.** Answer yes: a local “undo history” of the whole folder. Nothing is uploaded.

The installer then shows a summary and asks “Create the hub now?”. If anything in the summary is wrong, type `n`, press Return, and paste the install line again. Otherwise press Return. The installer creates the folder, runs a self-test and prints your next steps. It changes nothing else on your computer except the installations you agreed to. Installing Claude Code shows no progress while it downloads; wait for it to finish.

If the installer stops halfway (a lost connection, a closed window, an error), nothing is left behind. Fix the problem it reports, if any, and run the same command again.

Step 2. Your first session (about 10 minutes)

1. **Open a new terminal window,** so that newly installed programs are found. Mac: press Cmd+N in Terminal. Windows: open Ubuntu again from the Start menu.

2. **Start the assistant in your hub.** At the end, the installer printed a start line. If you accepted the suggested folder, it is:

   ```
   cd ~/research-hub && claude
   ```

   Type it and press Return. (With Codex, the line ends in `codex` instead; use one assistant at a time.) **Use this same line every time you come back to the hub.** Typing just `claude` would start the assistant in the wrong folder, without the hub's rules.

3. **First-time screens in Claude Code.** These appear only once.
   - "Choose the text style that looks best with your terminal": press Return.
   - "Select login method": choose **1. Claude account with subscription** and press Return.
   - Your browser opens. Sign in to your Claude account, approve the request, then come back to the terminal window. If no browser opens (common on Windows), the terminal shows a long web address. Select it with the mouse, copy it with Ctrl+Shift+C, and paste it into your browser. In a terminal, Ctrl+C on its own stops the program instead of copying. If the browser then shows a code, copy it, paste it into the terminal with Ctrl+Shift+V, and press Return.
   - If other one-time screens appear, press Return to accept the suggestion.
   - "Quick safety check: Is this a project you created or one you trust?" Press the **down arrow** to choose **"Yes, I trust this folder"**, then press Return. Return on its own chooses "No, exit" and closes Claude Code; if that happens, type the start line again. First check that the folder shown under "Accessing workspace" ends in `research-hub` (or the folder you chose); if it shows another folder, choose "No, exit" and use the start line. The same screen warns that the folder pre-approves some commands and adds a folder to the workspace. Those are the hub's own commands and your archive folders, so this is expected. Until you trust the folder, the hub's settings, including the protection of your archives, are ignored.

4. **First-time screens in Codex.** At “Sign in with ChatGPT”, press Return and sign in in your browser. If no browser opens (common on Windows), choose **2. Sign in with Device Code** instead and follow the instructions it shows. If Codex asks “Trust this folder?” (it may not, if the installer already recorded this), choose **Trust and continue** and press Return. Its warning that folder settings can run code is expected: those settings are the hub’s own, which you just installed.

5. **Ask for a tour.** Paste this and press Return:

   > Read START_HERE.md and AGENTS.md, then give me a ten-line tour of this hub and suggest what I should do first.

Permission prompts. The assistant asks before it runs a command or changes a file. Each prompt is a short menu: use the arrow keys to choose, then press Return. Esc means no. A good policy:

- **approve** the hub’s own commands (`python3 tools/hub.py ...`), reading files, and edits to notes inside the hub;
- **think twice** about anything that installs software, deletes files, or reaches outside the hub folder;
- **decline** anything you did not ask for. The assistant will adjust;
- **if you cannot tell what a request does,** decline it, then type in the message box: “Explain in plain English what that would do and why.” Press Return, read the answer, and decide.

The hub’s own commands and saving to the local version history are pre-approved, so most prompts you see concern new programs or files. Sending anything to the internet with git is blocked outright.

To stop the assistant mid-task, press **Esc**. To leave, type `/exit` or press **Ctrl+C** twice. Nothing is lost, because the work is in the files.

Step 3. Calibrate on a lemma you know (about 20 minutes)

Before trusting the assistant with open problems, find out how it behaves in your area. Use a lemma whose proof you know.

First, a proof attempt.

> In project [name], start an investigation of this lemma: [full statement, with all hypotheses]. Search the hub first. Then prove it, marking each step as proved, assumed or conjectural, and list the published results you rely on with exact theorem numbers. Then have a fresh proof-checker audit the proof, including the steps that connect the lemmas.

Compare its proof with yours.

- Which steps are right?
- Where did it wave its hands, and did it label those steps honestly?
- Did the checker catch the weak points?
- Are its citations real, and do they say what it claims? Check one yourself.

Then a counterexample test. Choose a hypothesis you know is necessary.

> Use the counterexample-checker to show that [hypothesis H] is needed in this lemma: find an example satisfying all the other hypotheses where the conclusion fails. Check every hypothesis for the example.

This takes twenty minutes and tells you more about how far to rely on it than any description can.

Step 4. Bring in your earlier work (about 15 minutes)

Do this in two requests, so that you see the list before anything is recorded.

> In project [name], search the archives for the main results, conjectures, computations and failed approaches in my earlier work. Do not record anything yet. Give me a numbered list with exact file locations and what each one claims.

> Record items [1, 3, 4] as “reported” with evidence level “imported” and exact locations. Record item [5] as an obstruction (a failed approach) with the condition under which it would be worth reopening.

Then fix the project’s rules:

> Help me fill in the “Mathematical conventions” and “Success threshold” sections of my project’s AGENTS.md. Ask me one question at a time.

Step 5. Work on a real problem

A proving session usually runs like this.

Generate ideas.

> Give me five genuinely different strategies for [open question]. For each, give the mechanism, the first calculation that would test it, and where it is most likely to fail. Search the hub first so you do not repeat recorded failures. Label everything conjectural.

Find the bottleneck.

> In project [name], start an investigation of [question] using strategy [3]. Search first, fill in the NOTE's Question and Prior work sections, and tell me the one missing implication before doing any proof work.

Attack it with the proof team.

> Use the proof swarm on that missing implication. Aim for a proof or a rigorously checked partial result. Stop after 2 hours.

The main session develops the argument and dispatches helpers for narrow side questions. It freezes any candidate proof and has a fresh checker audit it. By default Claude Code uses at most three helper jobs per request, with one kept for review; Codex uses six, with two for review. At most two run at once. To change this, add "budget: 5 helper jobs" to the request.

Close the session. This is the step that makes the next session fast.

> Close out this session following AGENTS.md, then give me a five-line summary of what changed.

Check the memory. Quit (type `/exit`), start a fresh session with your start line, and ask:

> Where does project [name] stand?

It picks up exactly where you stopped, from the files rather than from the conversation.

More ready-made requests are in PROMPTS.md, which is also in your hub.

Your first week

Day	Do this	End with
1	Install, first session, calibrate on a known lemma (Steps 1–3)	close out
2	Import earlier work; fill in conventions and success threshold (Step 4)	close out
3	Brainstorm strategies for one open question; pick one; find the missing implication	close out
4	Proof swarm on that implication, time-boxed to one or two hours	close out
5	Read the review of whatever survived; ask “where do we stand?” in a fresh session	close out

---

Getting the most out of it

- **Ask for the missing step, not the theorem.** “Prove the conjecture” gets a sweeping, unreliable answer. A precise implication gets real work.
- **Write the conventions once,** in the project rules. Every helper inherits them.
- **Let it search your past first.** Your old notes often contain the closest attempt and the reason it failed.
- **Use time boxes.** “Work on this for 2 hours” gives a session a clear budget and keeps the last part for review and write-up.
- **Feed failures back.** A counterexample to a step is often a corrected hypothesis in disguise. Ask “what is the strongest statement that survives this example?”
- **Ask who checked what.** “Which steps are proved, assumed or conjectural?” and “What exactly did the checker verify?” are the two most useful questions you can ask.
- **Use both assistants if you can.** A proof written with one and audited by the other is the strongest internal check the hub offers.
- **Always close out.** A session that is not closed out is mostly lost.

How it works, briefly

One folder, plain files. Everything is readable text: the rules, the notes, the catalog of results. The conversation is disposable and the folder is the memory.

A rulebook read at every session. `AGENTS.md` tells both assistants to search before proposing, state all hypotheses, treat remembered theorems as unchecked leads, label unproved steps, and never post, submit or email anything without your instruction. You can edit it; it is plain English.

A catalog with honest labels. Each claim has an ID (such as `RCF-R007`) and keeps four questions separate:

Question	Recorded as
Is it true?	status: reported, conditional, verified, open, refuted, superseded
Who checked it, and how much?	evidence: imported, draft proof, computation reproduced, checked against the source, internally reviewed, externally reviewed
Is it new?	novelty: unassessed, known, subsumed, candidate
Does it advance the goal?	the project guide and topic map

“Verified” means that a reviewer who did not write the proof audited a frozen copy of it, and that the review is on file. The catalog also records what depends on what, so when a lemma turns out to be wrong, one command lists everything built on it. A small tool checks every change before it is saved.

One notebook per question. Each investigation has a `NOTE.md` with the exact question, the missing implication, prior attempts, control examples, the work with each step labelled, and what would reopen any approach that failed.

A small team with a referee. There are five helper roles: archive scout, source checker, proof explorer, counterexample checker and proof checker. Each gets one question and only the files it needs. The author of a proof never reviews it.

Also included:

- earlier work becomes searchable but untouchable;
- a `library/` folder for PDFs of papers;

- exact retention of computations (code, inputs, outputs and versions);
- a paper-writing guide that drafts an amsart paper from reviewed results and refuses unchecked citations.

Limits and cautions

The assistants can be wrong, including the reviewers. Two AI reviewers can share a blind spot. Treat "verified" as "survived an independent audit of the stated scope". Read the argument yourself before you rely on it, and seek human review for anything that matters.

You are the author. Journals, arXiv and funding bodies have policies on AI assistance, and these change. Check the policy wherever you submit, and disclose AI use as it requires.

Confidentiality. Files the assistant reads are sent to the AI company. Check your account's privacy and data-use settings. Never put refereeing material, manuscripts shared with you in confidence, student information or passwords into the hub.

Novelty. A search that finds nothing proves nothing, and the tools say so every time.

Cost and limits. Plans limit how much you can use per few hours. The defaults keep usage modest; long proof campaigns use more.

Software changes. Both assistants update often. The repository includes a short check, `tests/live_check.sh`, that asks your assistants whether they still load the hub correctly.

Troubleshooting

- **"command not found: claude" (or codex).** Open a new terminal window. If it persists, paste the line below, press Return, open a new window, and type your start line again.
  - Mac: `echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.zshrc`
  - Windows (Ubuntu) and Linux: `echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc`

- **The installer says Python is missing.** On a Mac, click **Install** if a window offers the "command line developer tools"; otherwise type `xcode-select --install` and press Return. On Ubuntu type `sudo apt install python3` and press Return, then your Ubuntu password (it stays invisible). When that finishes, run the installer again.
- **The browser did not open for the login.** Copy the web address shown in the terminal (select it, then Ctrl+Shift+C on Windows or Cmd+C on a Mac), paste it into your browser, and follow the instructions there.
- **Claude Code closed at the "Quick safety check".** Return on its own means "No, exit". Type your start line again (such as `cd ~/research-hub && claude`), press the down arrow to choose "Yes, I trust this folder", then press Return.
- **Claude Code says your plan does not include it.** Claude Code needs a paid plan (Pro or above); the free plan does not include it. After upgrading, type `/login` inside Claude Code.
- **Typing a password in Terminal shows nothing.** That is normal: the characters are hidden. Type the password and press Return.
- **The installer stopped halfway.** Nothing was created. Run the same command again.
- **The assistant cannot find START_HERE.md, or seems not to know the hub.** It was probably started in another folder. Type `/exit`, then use your start line, `cd ~/research-hub && claude` (or the one the installer printed).
- **"already exists and is not empty".** A hub is already in that folder. Choose a different folder name when the installer asks, or use the hub you have.
- **The assistant changed something it should not have.** If you kept a version history, ask: "Show me what changed since the last commit and undo the edit to [file]." The catalog also keeps automatic backups.
- **Something seems broken.** In the hub folder, run `python3 tools/hub.py doctor` and `python3 tools/hub.py check`, then paste the output to the assistant.
- **A new project or another folder of old work.** Ask: "Create a new project 'short-name' with prefix XY about [question]", or "Register [folder] as a read-only archive for project [name]."

Glossary

- **Terminal:** a window for typing commands.
- **Agent:** an AI model that can act on your computer (read, write and run programs) instead of only talking.
- **Session:** one conversation with an agent. The hub's files persist between sessions; the conversation does not.

- **Helper:** a second agent started for one narrow question, with a fresh memory.
- **Close out:** the end-of-session routine that records results, updates the notes and checks the catalog.
- **Hash:** a fingerprint of a file's exact contents, used to notice when cited evidence has changed. It says nothing about whether the evidence is correct.
- **git:** a program that keeps every past version of a folder so that changes can be undone. Everything stays on your computer.

Everything here is open source and plain text: the rules, the templates and one standard-library Python file. Read it, change it, and adapt it to your field: github.com/ilpenzo/math-research-hub.

This site is open source. Improve this page.